Lập trình API -
Đồ họa cơ sở



Nguyễn Thu Hường

Khoa CNTT – Đại học GTVT

Các vấn đề của chương

- Cấu trúc của GDI
  - Ngữ cảnh thiết bị
  - Các hàm GDI
  - Những nguyên thuỷ của GDI
- Vẽ các điểm và các đường
  - Vẽ các điểm ảnh, đoạn thẳng
  - Vẽ các đường elíp
  - Bút vẽ, các chế độ vẽ
- Vẽ các miền kín
  - Chổi quét
  - Vẽ các hình chữ nhật, các hình elíp, ...

GDI là gì?

- **GDI** - Graphics Device Interface - Giao diện với các thiết bị đồ hoạ
- Là một thành phần quan trọng của Windows chịu trách nhiệm về việc hiển thị đồ hoạ lên màn hình máy tính hoặc ra máy in
- Không chỉ các chương trình ứng dụng, mà ngay bản thân Windows cũng sử dụng GDI để hiển thị thực đơn, thanh cuốn, biểu tượng và con trỏ chuột

Cấu trúc của GDI

- Từ khía cạnh của một lập trình viên, GDI bao gồm:
  - Vài trăm hàm
  - Một số kiểu dữ liệu
  - Các macro
  - Các cấu trúc có liên quan

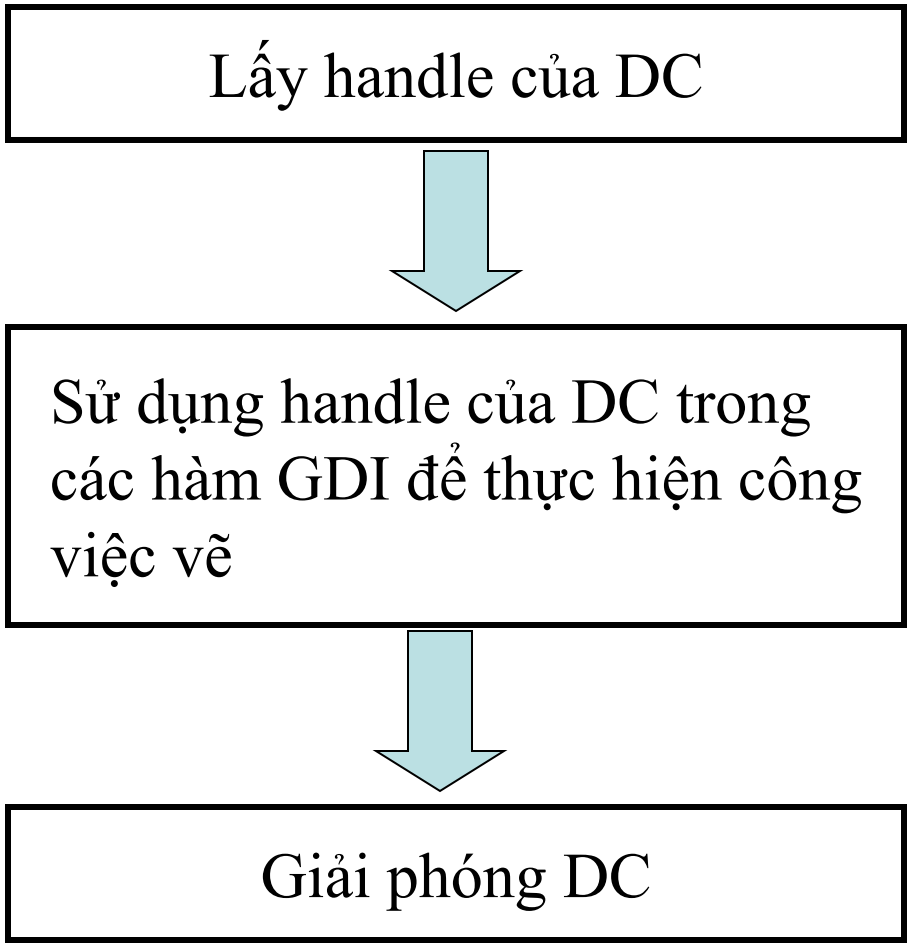
Ngữ cảnh thiết bị

- Ngữ cảnh thiết bị - Device Context (DC)
- Là một cấu trúc dữ liệu được duy trì bên trong GDI để định nghĩa các thuộc tính cho văn bản và hình ảnh được hiển thị ra màn hình hoặc máy in
- Được gắn với một thiết bị hiển thị cụ thể như màn hình hoặc máy in

Ngữ cảnh thiết bị

- Để đưa dữ liệu ra thiết bị hiển thị, ứng dụng chỉ cần sử dụng các hàm GDI để vẽ lên DC tương ứng
- Đối với màn hình, DC thông thường được gắn với một cửa sổ cụ thể
- Như hầu hết các đối tượng GDI, không thể truy cập trực tiếp tới DC, nhưng nó có thể được truyền vào các hàm GDI để:
  - Thực hiện công việc vẽ
  - Nhận thông tin về nó
  - Thay đổi 1 đối tượng bên trong nó

Các bước thực hiện công việc vẽ trong một chương trình



Lấy handle của DC

- Hai phương pháp lấy handle của DC
  - Lấy handle của DC trong khi xử lý thông điệp WM_PAINT
  - Lấy handle của DC trong khi xử lý các thông điệp khác

Lấy handle của DC – phương pháp 1


```
PAINTSTRUCT ps ;
HDC hdc ;
case WM_PAINT:
    hdc = BeginPaint (hwnd, &ps) ;
        [sử dụng các hàm GDI]
    EndPaint (hwnd, &ps) ;
    return 0 ;
```

Cấu trúc thông tin vẽ

```
typedef struct tagPAINTSTRUCT {
    HDC       hdc ;
    BOOL      fErase ;
    RECT      rcPaint ;
    BOOL      fRestore ;
    BOOL      fIncUpdate ;
    BYTE      rgbReserved[32] ;
} PAINTSTRUCT ;
```

- Chương trình chỉ có thể sử dụng 3 trường đầu tiên
- Các trường khác được sử dụng bên trong bởi Windows

Lấy handle của DC – phương pháp 1

- Thủ tục cửa sổ phải gọi BeginPaint và EndPaint theo cặp
- Hàm BeginPaint thông thường khiến nền của miền vô hiệu lực bị xoá (bởi chổi quét nền hiện thời) để chuẩn bị cho công việc tô vẽ

Lấy handle của DC – phương pháp 1

- Sau khi gọi hàm BeginPaint, toàn bộ miền thao tác trở thành có hiệu lực
  - **Chú ý:** Ứng dụng có thể làm hiệu lực hóa một miền hình chữ nhật bên trong miền thao tác bằng cách gọi hàm ValidateRect
- **Chú ý:** Công việc vẽ bị giới hạn trong phạm vi miền vô hiệu lực (miền cắt xén)

Lấy handle của DC – phương pháp 2

- Gọi GetDC để lấy handle và gọi ReleaseDC sau khi đã hoàn thành công việc vẽ:

```
hdc = GetDC (hwnd) ;
[sử dụng các hàm GDI]
ReleaseDC (hwnd, hdc) ;
```

- Chú ý:
  - Các hàm GetDC và ReleaseDC cần được gọi theo cặp
  - Đừng gọi GetDC trong một thông điệp và ReleaseDC ở thông điệp khác

Lấy handle của DC – phương pháp 2

- Các điểm khác biệt so với phương pháp 1
  - Handle của DC được trả ra từ GetDC có hình chữ nhật cắt xén bằng toàn bộ miền thao tác ⇒ Có thể vẽ ở bất cứ đâu trong miền thao tác
  - GetDC không làm hiệu lực hoá bất cứ miền vô hiệu lực nào cả

Phân loại các hàm GDI

- Các hàm để tạo và hủy một ngữ cảnh thiết bị
  - Ví dụ, các hàm BeginPaint và EndPaint trong thông điệp WM_PAINT
- Các hàm để nhận thông tin về ngữ cảnh thiết bị
  - Ví dụ, GetTextMetrics lấy kích thước của một font chữ
- Các hàm thực hiện công việc vẽ
  - Ví dụ, hàm TextOut hiển thị một văn bản

Phân loại các hàm GDI

- Các hàm thiết lập hoặc lấy những thuộc tính của một ngữ cảnh thiết bị
  - Ví dụ, SetTextColor thiết lập màu cho văn bản
  - Đối với tất cả các hàm Set thì đều tồn tại các hàm Get!!!
- Các hàm làm việc với các đối tượng GDI
  - Ví dụ, đối tượng **bút vẽ** cho biết độ rộng và kiểu của một đường (nét vẽ)

Những nguyên thuỷ của GDI

- Các đoạn thẳng và đường cong
- Các hình kín được tô màu
- Các ảnh bitmap
- Văn bản

Vẽ các điểm và các đường

- Giao diện GDI của Windows lợi dụng driver của các thiết bị đồ hoạ đầu ra để vẽ
- Về mặt lý thuyết:
  - Tất cả những gì mà driver của một thiết bị đồ hoạ cần để thực hiện công việc vẽ là hàm SetPixel và hàm GetPixel
  - Còn mọi thứ khác đều có thể được thực hiện bằng các hàm ở mức cao hơn
  - Ví dụ, vẽ một đoạn thẳng đơn giản chỉ cần GDI gọi hàm SetPixel nhiều lần với việc điều chỉnh các toạ độ x và y một cách thích hợp. **Tuy nhiên chậm!!!**
- Thực hiện một số công việc phức tạp ở mức driver của thiết bị đồ hoạ ➔ **Nhanh hơn nhiều**

Vẽ các điểm

- Hàm SetPixel thiết lập một điểm ảnh tại toạ độ x và y với một màu nào đó:
  - SetPixel (hdc, x, y, crColor) ;
  - Đối số cuối cùng thuộc kiểu COLORREF
    - typedef DWORD COLORREF;
    - 0x00bbggrr
    - Hãy sử dụng macro RGB:

    COLORREF RGB(BYTE *byRed*, BYTE *byGreen*, BYTE *byBlue*);
- Hàm GetPixel trả ra màu của điểm ảnh tại một vị trí nào đó: crColor = GetPixel (hdc, x, y) ;

Vẽ các đường

- Windows 98 hỗ trợ 7 hàm để vẽ các loại đường khác nhau:
  - BOOL LineTo(HDC hdc,int xPos,int yPos) - Vẽ một đoạn thẳng từ vị trí hiện hành đến (xPos,yPos) với màu hiện hành.
  - MoveToEx (hdc, xBeg, yBeg, NULL) ; - Thiết lập vị trí hiện thời mới ở tọa độ (xBeg, yBeg)
  - LineTo (hdc, xEnd, yEnd) ;
    - Vẽ đoạn thẳng từ vị trí hiện thời đến vị trí mới
    - Thiết lập vị trí hiện thời mới ở tọa độ (xEnd, yEnd)

Vẽ một đoạn thẳng

- Các hàm vẽ
  - PolyLine (hdc, const POINT *point,int nPoint) ;
    - Vẽ đường gấp khúc với tọa độ các điểm trong point
    - Số đỉnh là nPoint
  - Polygon(HDC hDC, const POINT *lpPoints, int nPoints)
    - Vẽ đa giác có nPoint đỉnh
    - Tọa độ các đỉnh được xác định ở point
- Lấy vị trí hiện thời:
  - GetCurrentPositionEx (hdc, &pt) ;
- **Chú ý**: Không cần phải thiết lập lại vị trí hiện thời khi bạn muốn vẽ một chuỗi các đoạn thẳng nối tiếp!!!

Vẽ một chuỗi các đoạn thẳng nối tiếp

- POINT apt[5] = {{100, 100}, {200, 100}, {200, 200}, {100, 200}, {100, 100 } ;
- **Chú ý:** Điểm cuối cùng trùng với điểm đầu tiên!!!
- **Cách 1**:

```
MoveToEx (hdc, apt[0].x, apt[0].y, NULL) ;
for (i = 1 ; i < 5 ; i++)
    LineTo (hdc, apt[i].x, apt[i].y) ;
                    /* Sử dụng vị trí hiện thời là điểm khởi đầu
                    Thiết lập vị trí hiện thời mới chính là điểm cuối
                    của đoạn thẳng cuối cùng*/
```

- **Cách 2**:

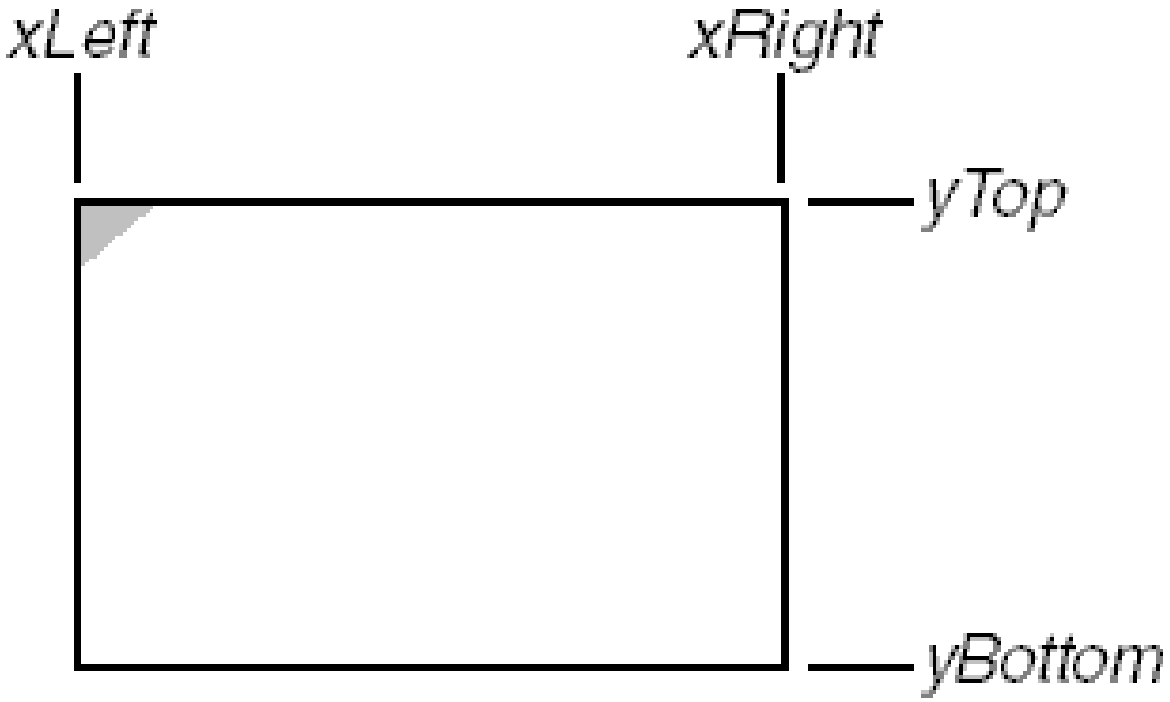
```
Polyline (hdc, apt, 5) ; //Không sử dụng hoặc thay đổi vị trí hiện thời
```

Vẽ hình chữ nhật và hình elíp

- Hai hàm Rectangle và Ellipse thực ra không hoàn toàn là các hàm vẽ các đường: Chúng đồng thời tô đầy miền kín bằng chổi quét hiện thời!!!
- Chổi quét mặc định là màu trắng đặc

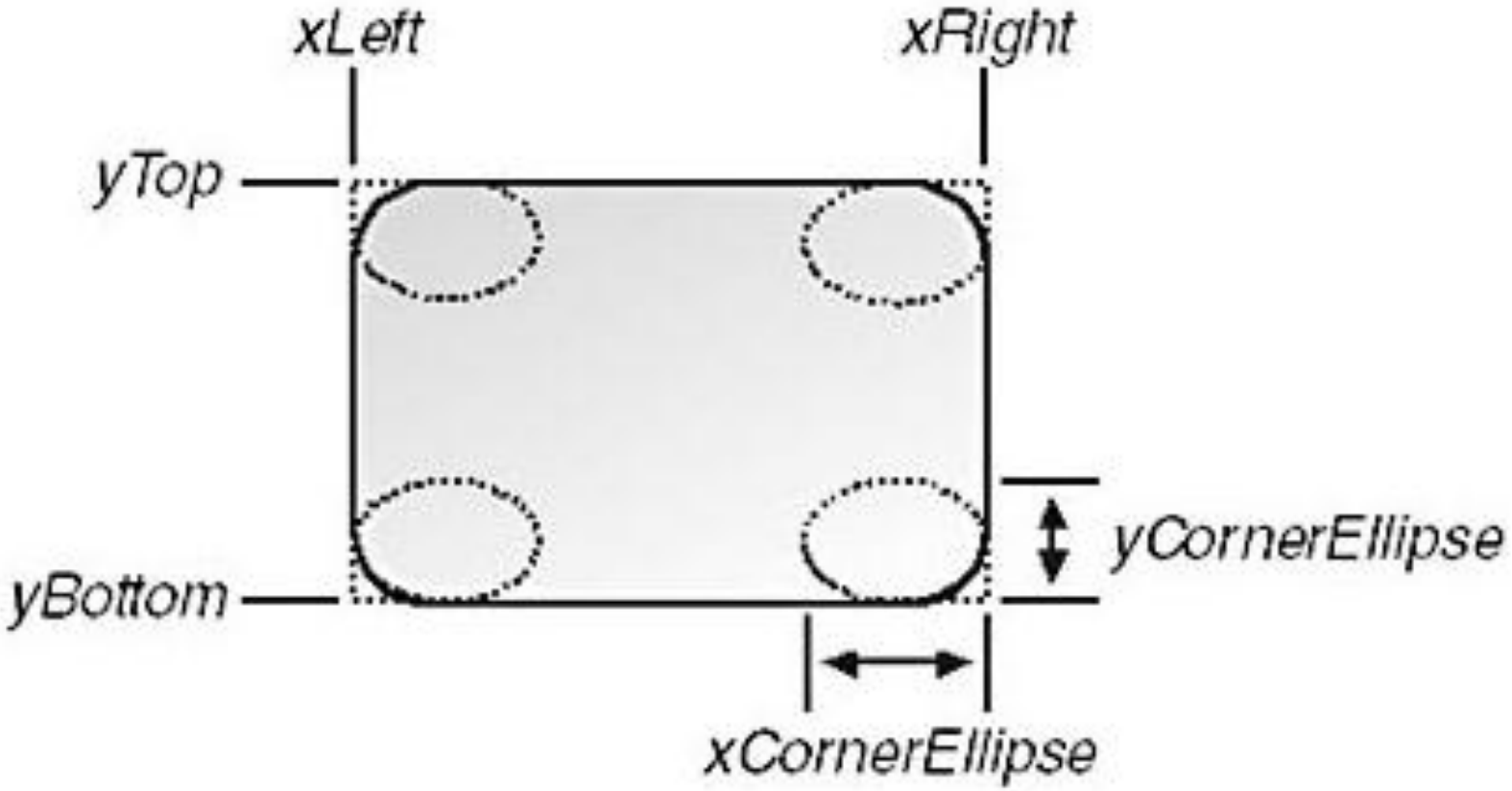
Vẽ hình chữ nhật

Rectangle (hdc, xLeft, yTop, xRight, yBottom) ;



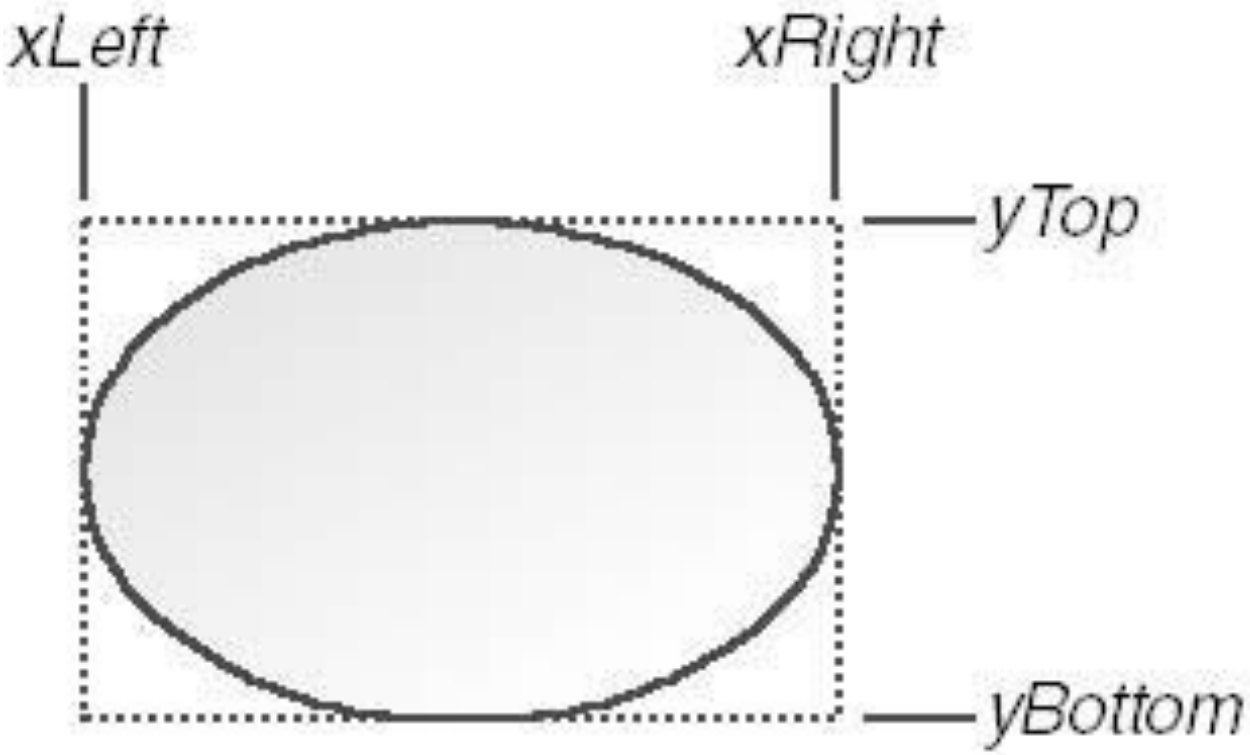
Vẽ hình chữ nhật với các góc trơn

RoundRect (hdc, xLeft, yTop, xRight, yBottom, xCornerEllipse, yCornerEllipse) ;



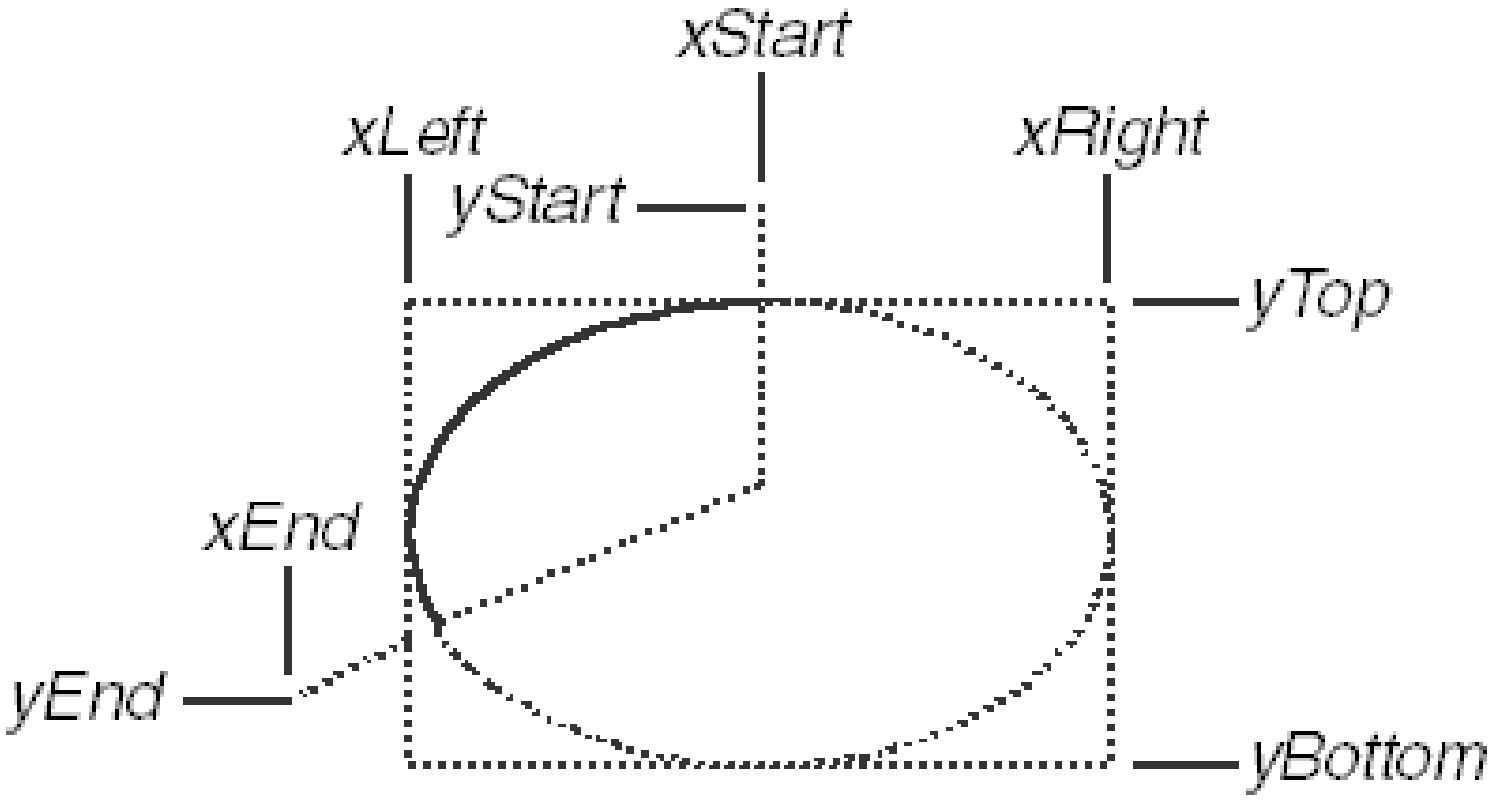
Vẽ elíp

Ellipse (hdc, xLeft, yTop, xRight, yBottom) ;



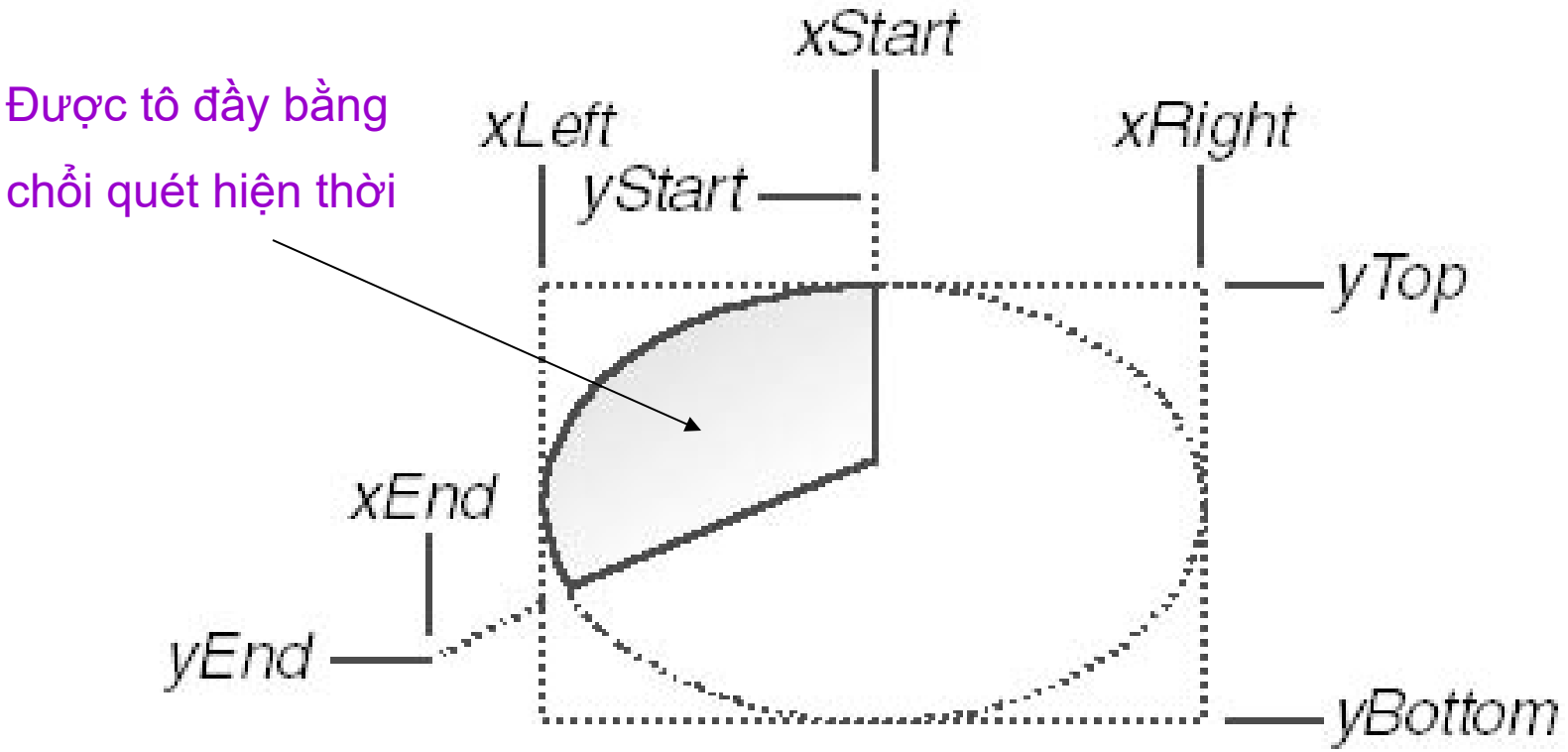
Hàm vẽ cung - Arc

Arc (hdc, xLeft, yTop, xRight, yBottom, xStart, yStart, xEnd, yEnd) ;



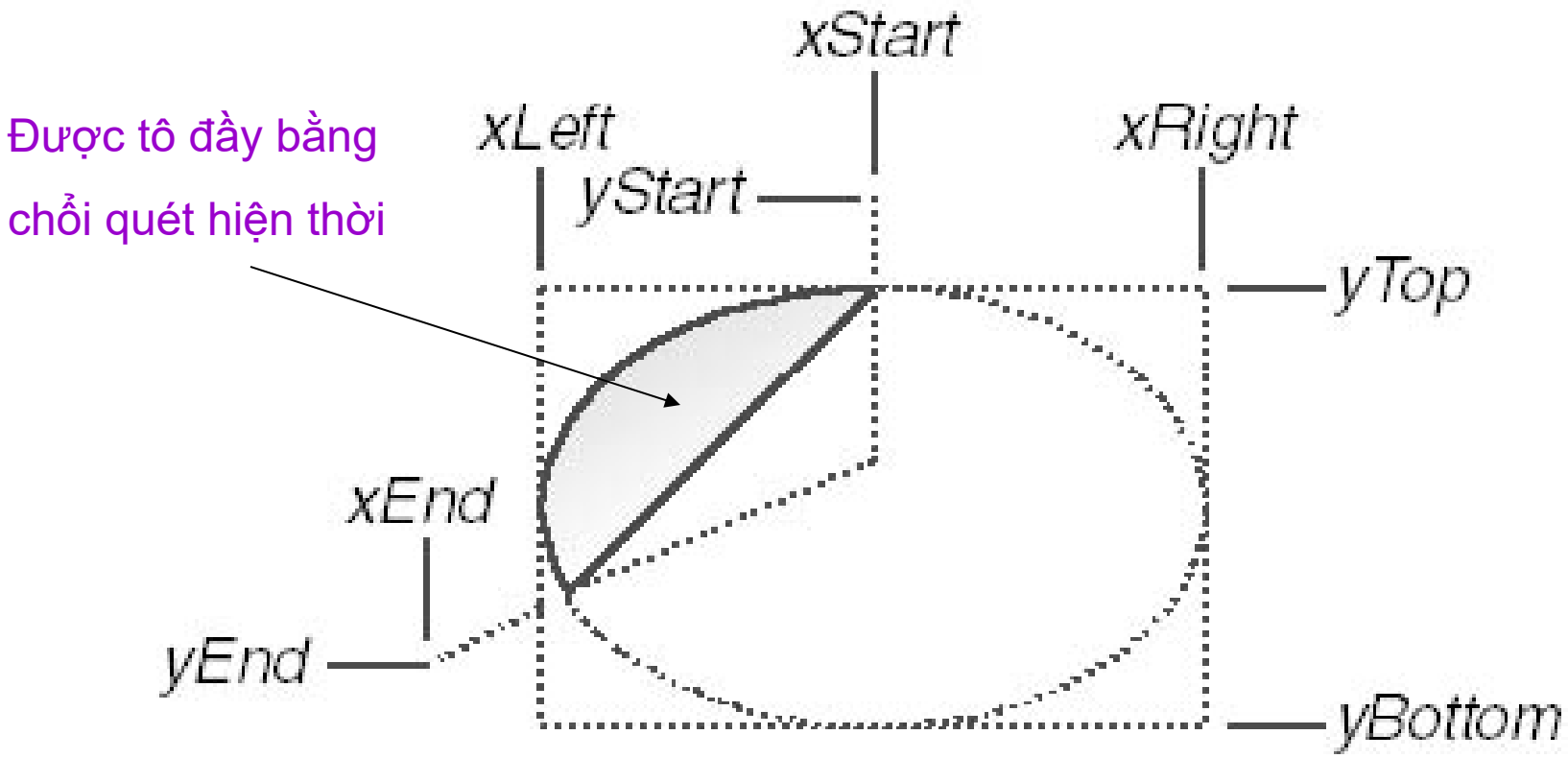
Hàm *hình quạt* - Pie

Pie (hdc, xLeft, yTop, xRight, yBottom, xStart, yStart, xEnd, yEnd) ;



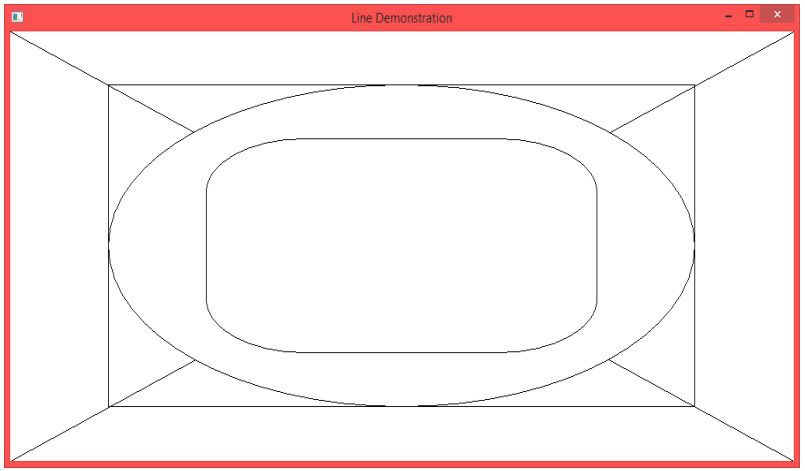
Hàm Chord

Chord (hdc, xLeft, yTop, xRight, yBottom, xStart, yStart, xEnd, yEnd) ;



Chương trình LineDemo

- Chương trình vẽ:
  - Một hình chữ nhật
  - Một hình elíp
  - Một hình chữ nhật với các góc trơn
  - Hai đoạn thẳng



- Nhận xét kết quả chương trình:
  - Các hàm vẽ các hình kín và thực ra là làm đầy chúng
  - Các đoạn thẳng bị khuất đằng sau hình elíp nên mới cho kết quả như vậy!!!
  - Khi cửa sổ thay đổi kích thước thì các hình này cũng bị thay đổi tương ứng.

Sử dụng bút vẽ

- Khi ta gọi bất cứ một hàm vẽ đường nào, Windows sẽ sử dụng **bút vẽ** hiện thời để vẽ
- Bút vẽ xác định:
  - Màu
  - Độ rộng
  - Kiểu mẫu
    - Đường liên tục
    - Đường chấm chấm
    - Hay đường gạch

Sử dụng bút vẽ

- Bút vẽ mặc định - BLACK_PEN
  - Đường liên tục
  - Màu đen
  - Độ rộng là 1 điểm ảnh
- Windows cung cấp 3 bút vẽ
  - BLACK_PEN
  - WHITE_PEN
  - NULL_PEN

  ⟸ GetStockObject()
- Có thể tạo một bút vẽ theo sở thích riêng!!!

Chọn bút vẽ

Cách 1:

```
HPEN hPen ;
hPen = (HPEN)GetStockObject (WHITE_PEN) ; //Lấy bút vẽ màu trắng đặc
SelectObject (hdc, hPen) ; //Chọn bút vẽ trắng đặc cho ngữ cảnh thiết bị hdc
```

Cách 2:

```
SelectObject (hdc, GetStockObject (WHITE_PEN)) ;
```

Tạo, chọn và huỷ các bút vẽ

- Tạo bút vẽ:
  - CreatePen(int style, int width, COLORREF color)
- Chọn bút vẽ cho ngữ cảnh thiết bị
  - SelectObject
- Huỷ bút vẽ
  - DeleteObject

Hàm CreatePen

hPen = CreatePen (iPenStyle, iWidth, crColor) ;

PS_SOLID	
PS_DASH	
PS_DOT	
PS_DASHDOT	
PS_DASHDOTDOT	
PS_NULL	
PS_INSIDEFRAME	

Nguyên tắc sử dụng các đối tượng GDI

- Phải giải phóng tất cả các đối tượng mà chúng ta đã tạo khi không còn cần sử dụng
- Đừng giải phóng các đối tượng GDI trong khi chúng đang được sử dụng trong một ngữ cảnh thiết bị

Ví dụ


```
static HPEN hPen1, hPen2, hPen3 ;
hPen1 = CreatePen (PS_SOLID, 1, 0) ;
hPen2 = CreatePen (PS_SOLID, 3, RGB (255, 0, 0)) ;
hPen3 = CreatePen (PS_DOT, 0, 0) ;

SelectObject (hdc, hPen2) ;
[ Các hàm vẽ đường ]
SelectObject (hdc, hPen1) ;
[ Các hàm vẽ đường ]
```

Giả sử rằng chương trình sử dụng 3 bút vẽ

- Một bút vẽ màu đen với độ rộng 1 điểm ảnh
- Một bút vẽ màu đỏ với độ rộng 3 điểm ảnh
- Một bút vẽ chấm chấm màu đen

- Trong thông điệp WM_DESTROY:

```
DeleteObject (hPen1) ;
DeleteObject (hPen2) ;
DeleteObject (hPen3) ;
```

Tạo và giải phóng bút vẽ


```
SelectObject(hdc, CreatePen (PS_DASH, 0, RGB (255, 0, 0)));
DeleteObject (SelectObject (hdc, GetStockObject (BLACK_PEN))) ;
```

- **Chú ý:** SelectObject trả ra handle của bút vẽ hiện thời (cũ) của ngữ cảnh thiết bị đồng thời thiết lập bút vẽ mới!!!

Tạo và giải phóng bút vẽ

- Phương pháp khác:

```
hPen = SelectObject (hdc, CreatePen (PS_DASH, 0, RGB (255, 0, 0))) ;
DeleteObject (SelectObject (hdc, hPen)) ;
```

- Lấy handle của bút vẽ hiện thời:

```
hPen = GetCurrentObject (hdc, OBJ_PEN) ;
```

Làm đầy các quãng ngắt

- Việc sử dụng các bút vẽ chấm chấm và gạch ngang phát sinh một câu hỏi: Cái gì ở các quãng ngắt giữa các dấu chấm và các nét gạch?
- Việc tô màu cho các quãng ngắt phụ thuộc vào 2 thuộc tính của ngữ cảnh thiết bị
  - Chế độ nền
  - Màu nền
- Chế độ nền mặc định là OPAQUE

Làm đầy các quãng ngắt

- Thay đổi màu nền
  - SetBkColor (hdc, crColor) ;
- Lấy màu nền hiện thời
  - GetBkColor()
- Ngăn không cho Windows làm đầy các quãng ngắt bằng việc thay đổi chế độ nền sang TRANSPARENT:
  - SetBkMode (hdc, TRANSPARENT) ;

Vẽ các miền kín được tô đầy

Hàm	Hình
Rectangle	Hình chữ nhật với các góc vuông
Ellipse	Elíp
RoundRect	Hình chữ nhật với các góc tròn
Chord	Cung elíp với 2 điểm đầu và cuối của cung được nối với nhau
Pie	Cung elíp với 2 điểm đầu và cuối của cung nối với tâm elíp
Polygon	Hình đa giác
PolyPolygon	Một chuỗi các đa giác

Chổi quét - Brush

- Sử dụng chổi quét hiện thời trong ngữ cảnh thiết bị để tô bên trong các miền kín
- Chổi quét là một ảnh bitmap nhỏ kích thước 8×8 điểm ảnh mà được lặp đi lặp lại theo chiều ngang và chiều dọc
- Khi Windows sử dụng phép phối màu để hiển thị nhiều màu hơn so với khả năng mà một màn hình có thể hiển thị thì thực tế nó sử dụng chổi quét để hiển thị màu cần thiết!!!

Chổi quét

- Trên hệ thống đơn sắc, Windows có thể sử dụng phép phối màu giữa các điểm đen và trắng để tạo 64 sắc thái của màu xám
- Chính xác hơn, Windows có thể tạo 64 chổi quét đơn sắc khác nhau
  - Đối với màu đen thuần khiết, tất cả các bit trong ảnh bitmap 8×8 đều là 0
  - Một bit trong số 64 bit là 1 (tức là màu trắng) sẽ cho sắc thái đầu tiên của màu xám
  - 2 bit là màu trắng sẽ cho sắc thái thứ 2 của màu xám
  - Và cứ như thế, đến khi tất cả các bit trong ảnh bitmap 8×8 là 1 sẽ cho kết quả là màu trắng tinh khiết

Tạo chổi quét

- Windows có 5 hàm cho phép tạo các chổi quét lôgíc
- Hàm đầu tiên
  - hBrush = CreateSolidBrush (crColor) ;

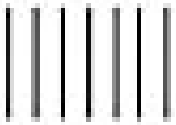
Tạo chổi quét

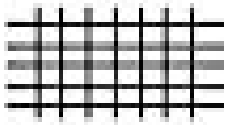
- Tạo một chổi quét với các đường gạch:

```
hBrush = CreateHatchBrush (iHatchStyle, crColor) ;
```

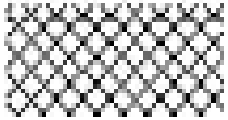
HS_HORIZONTAL 

HS_BDIAGONAL 

HS_VERTICAL 

HS_CROSS 

HS_FDIAGONAL 

HS_DIAGCROSS 

Các hàm làm việc với chổi quét

- SelectObject - Lựa chọn một bút vẽ lôgíc
- DeleteObject - Giải phóng một bút vẽ lôgíc
- GetObject - Lấy thông tin về bút vẽ lôgíc